

Diego Preciado Rodríguez
Lúa Izel Soto Sánchez
Alejandra Peña Andrade
Pedro Emiliano Ortega Ortiz
Jorge Diego Valdivieso Saure

RIFTFORGE TCG

DOCUMENTO DE DISEÑO Y EJECUCIÓN

1. PROPOSITO DEL PROYECTO

RiftForge TCG es una simulación de duelo local desarrollada en Java. El objetivo es demostrar el uso de tipos de datos y estructuras enlazadas construidas con nodos propios. El proyecto no utiliza las clases LinkedList, Stack ni Queue de Java para implementar la lógica solicitada.

2. TIPO DE DATO ABSTRACTO CARTA

La clase Card representa el Tipo de Dato Abstracto Carta. Agrupa los datos y el comportamiento de una carta del juego.

Datos primitivos: manaCost, attack y health son valores enteros.

Datos complejos: name es un valor String. También se usan Element y CardType para clasificar la carta.

Comportamiento: el constructor valida que los datos sean correctos, attackWithBonus calcula el ataque con un efecto de campo y toString presenta una descripción legible.

Ejemplo: Caballero de Fuego tiene coste 3, ataque 5 y vida 6. Si el efecto Fuego Igneo está activo, recibe un bono de 2 puntos de ataque.

Diego Preciado Rodríguez
Lúa Izel Soto Sánchez
Alejandra Peña Andrade
Pedro Emiliano Ortega Ortiz
Jorge Diego Valdivieso Saure

3. ESTRUCTURAS DE DATOS IMPLEMENTADAS

3.1 Lista simplemente enlazada. Catalogo de cartas.

La clase SinglyLinkedList utiliza nodos con una referencia al siguiente nodo. GameEngine usa esta estructura para guardar el catalogo. Permite registrar una carta, buscar una carta, eliminar una carta y recorrer todas las cartas. La opcion C del programa muestra el recorrido del catalogo.

3.2 Lista doblemente enlazada. Historial de jugadas.

La clase DoublyLinkedList utiliza nodos con referencias al nodo siguiente y al anterior. Cada turno crea un BattleEvent y lo agrega al final del historial. El metodo forward permite revisar las jugadas desde la primera hasta la ultima. El metodo backward permite revisarlas desde la mas reciente hasta la primera. La opcion H muestra ambos recorridos.

3.3 Lista circular. Ciclo de efectos de campo.

La clase CircularLinkedList mantiene el ultimo nodo conectado con el primero. El juego registra cuatro efectos: Fuego Igneo, Tormenta Electrica, Vacio Abisal y Marea Cristalina. El efecto activo permanece dos turnos. Al terminar el segundo turno, rotate avanza al siguiente efecto. Despues de Marea Cristalina el ciclo vuelve a Fuego Igneo.

3.4 Pilas. Mazo de robo y mazo de descarte.

Diego Preciado Rodríguez
Lúa Izel Soto Sánchez
Alejandra Peña Andrade
Pedro Emiliano Ortega Ortiz
Jorge Diego Valdivieso Saure

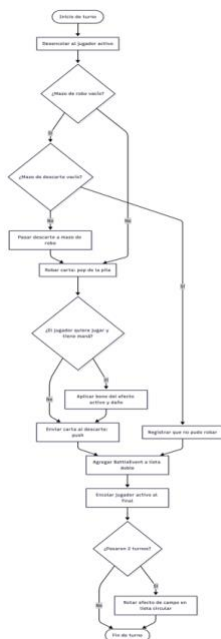
Cada jugador tiene dos instancias de LinkedStack. La pila deck es el mazo de robo y la pila graveyard es el mazo de descarte. Robar una carta usa pop. Descartar o jugar una carta usa push. Cuando el mazo de robo esta vacio y hay cartas descartadas, el motor pasa las cartas del descarte al mazo de robo antes de continuar.

3.5 Cola circular. Turnos de jugadores.

La clase CircularQueue administra los turnos. El jugador del frente se desencola para realizar su turno y luego se encola al final. El siguiente jugador queda al frente. Esta operacion mantiene el orden de turnos de manera repetitiva.

4. DIAGRAMA DE FLUJO DEL TURNO

El diagrama de flujo entregado en DOCUMENTACION.md representa la siguiente secuencia, que coincide con GameEngine.playTurn.



Diego Preciado Rodríguez
Lúa Izel Soto Sánchez
Alejandra Peña Andrade
Pedro Emiliano Ortega Ortiz
Jorge Diego Valdivieso Saure

5. RESPONSABILIDAD DE LAS CLASES

Main crea la partida de ejemplo, registra cartas y efectos, y procesa las opciones de consola.

GameEngine coordina las reglas del turno, el robo, el dano, el historial y la rotacion de efectos.

Card representa una carta y calcula su ataque con bono.

Player guarda nombre, vida, mana, mazo de robo y mazo de descarte.

Rift representa un efecto de campo inmutable.

BattleEvent representa una jugada inmutable del historial.

Node es el nodo interno reutilizado por las estructuras enlazadas.

SinglyLinkedList implementa el catalogo.

DoublyLinkedList implementa el historial bidireccional.

CircularLinkedList implementa el ciclo de efectos.

LinkedStack implementa las pilas de cartas.

CircularQueue implementa el orden de iniciativa.

ConsoleRenderer concentra la presentacion por consola sin incluir reglas del juego.

6. CASOS DE USO Y RESULTADOS

Diego Preciado Rodríguez
Lúa Izel Soto Sánchez
Alejandra Peña Andrade
Pedro Emiliano Ortega Ortiz
Jorge Diego Valdivieso Saure

Caso 1. Recorrido del catalogo. Se presiona C al iniciar. Resultado esperado: se muestran las cuatro cartas registradas. Resultado observado: las cuatro cartas se muestran en orden inverso al registro porque la lista inserta al inicio.

Caso 2. Dos turnos consecutivos. Se presiona J dos veces. Resultado esperado: cada jugador roba una carta, la carta se descarta, se registra el evento y el turno cambia de jugador. Resultado observado: Ana invoca Monstruo de Agua e inflige 3 a Beto. Beto invoca Caballero de Fuego e inflige 7 a Ana por el bono de Fuego Igneo. Despues del segundo turno, el siguiente efecto es Tormenta Electrica.

Caso 3. Historial en ambos sentidos. Se presiona H despues de dos turnos. Resultado esperado: se muestran dos jugadas desde la primera hasta la ultima y luego desde la ultima hasta la primera. Resultado observado: el programa muestra ambos ordenes correctamente.

Caso 4. Mazo de robo agotado. Resultado esperado: las cartas del descarte se reutilizan como mazo de robo. Resultado observado: `recycleDeckIfNeeded` transfiere cada carta de una pila a la otra antes del robo.

Caso 5. Ambas pilas vacias. Resultado esperado: el programa no falla y registra que el jugador no puede robar. Resultado observado: se agrega un `BattleEvent` y el turno pasa al siguiente jugador.

7. EJEMPLO DE EJECUCION

Diego Preciado Rodríguez
Lúa Izel Soto Sánchez
Alejandra Peña Andrade
Pedro Emiliano Ortega Ortiz
Jorge Diego Valdivieso Saure

Turno 1. Ana invoca Monstruo de Agua e inflige 3 a Beto.

Turno 2. Beto invoca Caballero de Fuego e inflige 7 a Ana.

Historial hacia adelante: primero aparece el turno 1 y despues el turno 2.

Historial hacia atras: primero aparece el turno 2 y despues el turno 1.

8. INSTRUCCIONES DE EJECUCION

Desde la raiz del proyecto, compile y ejecute con los siguientes comandos.

```
javac -d out java/src/riftforge/structures/Node.java y los demas archivos fuente.
```

```
java -cp out riftforge.app.Main
```

Opciones disponibles: J para jugar, D para descartar, C para catalogo, H para historial, Q para reiniciar y X para terminar.

9. COMPLEJIDAD

Las operaciones push, pop, enqueue, dequeue, addLast y rotate son de tiempo constante. Buscar, eliminar y recorrer elementos son operaciones lineales. La carga inicial de los cuatro efectos puede recorrer la lista circular para mantener el orden configurado, lo cual es adecuado porque se realiza solo al iniciar la partida.